

Introducción al bucle `for` en Python

El bucle `for` sirve para recorrer (iterar) una secuencia de elementos: por ejemplo, una lista, una cadena, una tupla o un rango de números.

✦ En lugar de repetir mientras se cumple una condición (como en `while`), el `for` va pasando automáticamente por cada elemento de algo.

Ejemplo simple

python

 Copiar código

```
for numero in [1, 2, 3, 4, 5]:  
    print("Número:", numero)
```

Salida:

makefile

 Copiar código

```
Número: 1  
Número: 2  
Número: 3  
Número: 4  
Número: 5
```

✦ Aquí el `for` va "sacando" cada número de la lista y lo guarda en `numero` en cada iteración.

🟡 En Java (y otros lenguajes como C o C++)

El bucle `for` tiene una estructura clásica con tres partes dentro de los paréntesis:

```
java

for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Ahí tienes:

- 1 Inicialización: `int i = 1`
- 2 Condición: `i <= 5`
- 3 Incremento: `i++`

El bucle se controla **manualmente** con esas tres partes.

🟢 En Python, NO se usa esa sintaxis

En Python **no existe** esa forma con tres secciones separadas por `;`.

👉 En lugar de eso, Python usa una versión más simple y "automática" basada en **secuencias**:

```
python

for i in range(1, 6):
    print(i)
```

💡 Esto hace lo mismo que el `for` clásico de Java:

- `range(1, 6)` genera los números 1, 2, 3, 4, 5
 - `i` va tomando cada valor automáticamente
 - No necesitas escribir `i++` (Python lo hace por ti internamente)
-

🔍 Si quisieras algo más "manual"

Puedes hacerlo con `while`, que en Python sí permite el control completo:

```
python

i = 1
while i <= 5:
    print(i)
    i += 1
```

👉 Esto se parece más al `for` tradicional de Java.



WHILE vs FOR

Comparación clara

Aspecto	<code>while</code>	<code>for</code>
Necesitas inicializar el contador	✓ Sí (<code>i = 1</code>)	✗ No (lo hace <code>range</code>)
Necesitas incrementar manualmente	✓ Sí (<code>i += 1</code>)	✗ No
Condición explícita	✓ <code>i <= 5</code>	✗ (implícita en <code>range</code>)
Riesgo de bucle infinito	😬 Sí	🚫 No
Ideal para...	Bucles dependientes de condiciones	Recorrer secuencias o rangos

La función `range()`

Sirve para **generar una secuencia de números**.

Puedes usarla de **tres formas diferentes**, según lo que necesites.

1 `range(fin)`

👉 Genera números **desde 0 hasta fin - 1**.

python

```
for i in range(5):  
    print(i)
```

 **Salida:**

```
0  
1  
2  
3  
4
```

💡 Por defecto:

- Empieza en 0
- Va de 1 en 1
- **No incluye** el número final

🟡 2 range(inicio, fin)

👉 Genera números desde inicio hasta fin - 1.

```
python

for i in range(2, 6):
    print(i)
```

🖨 Salida:

```
2
3
4
5
```

💡 Aquí ya defines tú el punto de partida.

3 **range(inicio, fin, paso)**

 Genera números desde inicio hasta fin - 1, de "paso" en "paso".

python

```
for i in range(1, 10, 2):  
    print(i)
```

 **Salida:**

```
1  
3  
5  
7  
9
```

 El tercer número indica de cuánto aumenta (o disminuye) cada vez.

▼ También puede ir hacia atrás

Si el `paso` es **negativo**, cuenta hacia atrás 🗨️

```
python
```

```
for i in range(10, 0, -2):  
    print(i)
```

🖨️ Salida:

```
10  
8  
6  
4  
2
```

💡 Empieza en 10, y cada vez resta 2.

Resumen rápido del uso de `for` y `range()`

Forma con <code>for</code>	Qué hace	Ejemplo	Salida
<code>for i in range(fin):</code>	Recorre de 0 a fin - 1	<code>for i in range(5):</code>	0 1 2 3 4
<code>for i in range(inicio, fin):</code>	Recorre de inicio a fin - 1	<code>for i in range(2, 6):</code>	2 3 4 5
<code>for i in range(inicio, fin, paso):</code>	Recorre de inicio a fin - 1 saltando de "paso" en "paso"	<code>for i in range(1, 10, 2):</code>	1 3 5 7 9
<code>for i in range(inicio, fin, paso_negativo):</code>	Recorre hacia atrás cuando el paso es negativo	<code>for i in range(10, 0, -2):</code>	10 8 6 4 2

Truco útil:

Si quieres ver rápidamente qué números genera un `range`, conviértelo en lista:

python

 Copiar código

```
print(list(range(1, 10, 2)))
```

Salida:

csharp

 Copiar código

```
[1, 3, 5, 7, 9]
```

EJERCICIO 1: Suma de los números del 1 al 5 con `while` y con `for`. Crea un programa que calcule la suma de los números del 1 al 5, 2 veces, la primera utilizando un bucle `while` y la segunda utilizando un bucle `for`, para comparar el funcionamiento de ambos tipos de bucles.

EJERCICIO 2: Ejemplos prácticos del uso de `range()` en Python

Objetivo:

Entender cómo funciona la función `range()` y aprender a generar secuencias de números en diferentes situaciones usando sus distintas formas.

Instrucciones:

El programa mostrará ejemplos de las principales formas de usar `range()`:

1) `range(fin)`

- Genera números desde 0 hasta fin - 1.

- 2) `range(inicio, fin)`
 - Genera números desde 'inicio' hasta 'fin - 1'.
- 3) `range(inicio, fin, paso)`
 - Permite indicar un incremento o salto entre valores.
- 4) `range(inicio, fin, paso negativo)`
 - Cuenta hacia atrás cuando el paso es negativo.
- 5) `list(range(...))`
 - Convierte los objetos range en listas visibles para mostrar sus valores.

Aprendizaje:

Este ejercicio sirve para visualizar cómo `range()` crea secuencias de números en distintos escenarios y cómo puede utilizarse en bucles for. También se comprende que el valor final de range nunca se incluye.

EJERCICIO 3 — Suma de números pares hasta un límite que indique el usuario. Aquí nos damos cuenta de que, si ingresamos 8, nos suma 2+4+6, si quisieramos que llegara hasta el 8 deberíamos sumar 1, al límite o número que nos diga el usuario.

🧠 ¿Por qué Python no llega hasta el final en los rangos y las listas?

("Inicio incluido, final excluido")

💡 Ventaja	🌱 Explicación	📄 Ejemplo	🎯 Resultado / Beneficio
1 Cálculo sencillo de longitud	La cantidad de elementos de un rango o una porción siempre es <code>fin - inicio</code> .	<code>range(2, 8) → 8 - 2 = 6</code> elementos	<code>[2, 3, 4, 5, 6, 7]</code> (6 valores exactos)
2 Fácil de encadenar sin huecos ni solapamientos	Dos rangos consecutivos se conectan perfectamente sin repetir ni saltar números.	<code>range(0, 5)</code> y <code>range(5, 10)</code>	<code>[0,1,2,3,4] + [5,6,7,8,9]</code> encajan perfecto ✅
3 Compatible con los índices de listas	Las listas usan índices desde 0 hasta <code>len(lista)-1</code> . Un rango con <code>range(len(lista))</code> genera exactamente esos índices válidos.	<code>lista = [10, 20, 30, 40, 50]</code> <code>for i in range(len(lista)):</code>	Accede a <code>lista[0]</code> a <code>lista[4]</code> sin error ⚙️
4 Coherencia con slicing (<code>[:]</code>)	Python usa la misma regla para trozos de listas o strings.	<code>"python"[0:3]</code>	<code>'pyt'</code> (coge hasta el índice 2, no el 3)
5 Más seguro y predecible	Evita errores por intentar acceder al índice "fuera de rango".	<code>range(len(lista))</code> con 5 elementos	Nunca intenta <code>lista[5]</code> ❌

EJERCICIO 4: Contar múltiplos de 3 hasta un número dado por el usuario

Pedir al usuario un número límite y determinar cuántos múltiplos de 3 hay entre 1 y ese número. Además, mostrar por pantalla todos los múltiplos en una misma línea.

EJERCICIO 5: Calcular el factorial de un número

Pedir al usuario un número entero positivo y calcular su factorial.

El factorial de un número ($n!$) es el producto de todos los enteros positivos desde 1 hasta n .

Por ejemplo:

$$5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$$

EJERCICIO 6 — Calcular la suma de los cuadrados de los primeros N números

Suma de los cuadrados de los primeros N números

Pedir al usuario un número entero N y calcular la suma de los cuadrados de todos los números desde 0 hasta $N - 1$.

Por ejemplo:

$$\text{Si } N = 5 \rightarrow 0^2 + 1^2 + 2^2 + 3^2 + 4^2 = 30$$

EJERCICIO 7: Números divisores de un número dado

Pedir al usuario un número entero positivo y mostrar todos los números que son divisores exactos de ese número (es decir, que lo dividen sin dejar resto).

Por ejemplo:

Si el usuario introduce 12 $\rightarrow$ los divisores son 1, 2, 3, 4, 6 y 12.

⚙ Instrucciones especiales dentro del `for`

Sí 😊, al igual que con `while`, también puedes usar:

■ `break`

➡ Sale completamente del bucle antes de que termine.

python

```
for i in range(1, 10):  
    if i == 5:  
        break  
    print(i)
```

🖨 Salida:

```
1  
2  
3  
4
```



👉 Cuando `i == 5`, se rompe el bucle.

EJERCICIO 8: Determinar si un número es primo

Pedir al usuario un número entero positivo y comprobar si es un número primo.
Un número es primo si solo tiene dos divisores: 1 y él mismo.

Por ejemplo:

- 7 es primo → solo divisible por 1 y 7
- 12 no es primo → divisible por 1, 2, 3, 4, 6, 12

EJERCICIO 9: Pedir al usuario un número límite y sumar los números naturales desde 1 en adelante hasta que la suma acumulada supere ese límite.
En ese momento, el programa se detiene con 'break' y muestra el número que hizo que la suma sobrepasara el límite.

continue

➔ Salta al siguiente elemento sin ejecutar el resto del código.

python

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```

🖨 Salida:

```
1  
2  
4  
5
```

👉 Se salta el 3, pero sigue con el resto.

EJERCICIO 10: Pedir al usuario un número límite y calcular la suma de todos los números impares desde 1 hasta ese límite.

Los números pares se saltan usando la instrucción 'continue'.

✓ `else` (también funciona en `for`)

El bloque `else` se ejecuta solo si el bucle termina normalmente (sin `break`):

python

```
for i in range(5):  
    print(i)  
else:  
    print("Bucle completado.")
```

🔴 Salida:

```
0  
1  
2  
3  
4  
Bucle completado.
```

Si el bucle acaba normal, sin `break`, da igual que esté el `else`, pq el `print` se va a ejecutar;